PRODUCT REQUIREMENTS DOCUMENT · V 0.1

FOOTPRINTX

Digital Footprint Intelligence Platform

A privacy-first, open-source platform that turns scattered OSINT signals — known breaches, account registrations, and public web references — into one evidence-backed Digital Exposure Report with confidence scoring, risk analysis, and remediation guidance.

Version 0.1 (MVP baseline)

Status Open source · defensive security tool

Date October 2026

NEXT.JS · FASTAPI · POSTGRESQL · REDIS

PRIVACY-FIRST · CONSENT-GATED · SELF-ASSESSMENT

Table of Contents

1. Executive Summary 1

1.1 Product overview . . . 1

1.2 The problem . . . 1

1.3 Vision and differentiator . . . 1

1.4 Primary goals . . . 2

2. Product Vision, Principles & Non-Goals 2

2.1 Core principle: self-assessment, not surveillance . . . 2

2.2 Default workflow . . . 2

2.3 Design principles . . . 3

2.4 Non-goals . . . 3

3. Target Users & MVP Scope 4

3.1 Personas . . . 4

3.2 MVP v0.1 scope . . . 4

4. System Architecture 5

4.1 MVP architecture overview . . . 5

4.2 Scanner framework . . . 6

4.3 Unified finding schema . . . 7

4.4 Event-driven scanning . . . 8

5. Technology Stack & Licensing 8

5.1 Frontend . . . 8

5.2 Backend . . . 8

5.3 Job system and data stores . . . 9

5.4 Stack summary . . . 9

5.5 Licensing caveat . . . 10

6. Scanner & Provider Integrations 10

6.1 Integration overview . . . 10

6.2 XposedOrNot (default breach provider) . . . 10

6.3 Have I Been Pwned (optional secondary) . . . 11

6.4 Holehe (email account presence) . . . 11
6.5 Sherlock and WhatsMyName (username discovery) . . . 11
6.6 Public web and certificate transparency (Phase 2) . . . 11
7. Correlation, Confidence & Risk Scoring 12
7.1 Identity correlation engine . . . 12
7.2 Found is not confirmed . . . 12
7.3 Risk scoring model . . . 13
8. Product Experience 14
8.1 Scan lifecycle and progress . . . 14
8.2 Results dashboard . . . 14
8.3 Breach presentation rules . . . 15
8.4 Accounts and public exposure . . . 15
8.5 Recommendations engine . . . 15
8.6 Scan modes . . . 15
8.7 Privacy Mode . . . 16
8.8 Local-only mode . . . 16
9. API, Data & Security Design 16
9.1 API surface . . . 16
9.2 Data model and identifier handling . . . 17
9.3 Source attribution . . . 17
9.4 Rate limiting . . . 17
9.5 Scanner health system . . . 18
9.6 Security requirements . . . 18
9.7 Audit logging . . . 18
10. Delivery Roadmap 18
10.1 Phased plan . . . 18
10.2 Monitoring and notifications . . . 19
10.3 v1.0 definition . . . 19

1. Executive Summary

1.1 Product overview

FootprintX is an open-source digital footprint intelligence platform that helps a person discover and understand their own publicly observable exposure across the internet. Given a single identifier such as an email address or a username, the platform dispatches a set of consent-gated OSINT scanners, normalizes their heterogeneous output into one unified finding schema, correlates the results into identity clusters, computes an exposure score, and renders an evidence-backed Digital Exposure Report with concrete remediation steps. The product is defensive by design: it answers what is publicly discoverable about the identifier's owner, never how to investigate someone else without authorization.

This document is the product requirements specification for FootprintX v0.1. It consolidates the product vision, target users, MVP scope, system architecture, technology stack, scanner integrations, intelligence design, product experience, API contracts, security posture, and a phase-by-phase delivery roadmap. It is written for engineers, designers, and open-source contributors who intend to build the first release, and it deliberately records both what the product will do and what it will refuse to do.

1.2 The problem

A person's digital identity is scattered across social networks, developer platforms, forums, public websites, search-engine indexes, old and forgotten accounts, known data breaches, public documents, profile aggregators, and historical references. There is presently no single, trustworthy, open-source place where an individual can answer three simple questions: what information about me is publicly discoverable, which accounts may belong to me, and has my information appeared in known breaches. Existing tools answer fragments of these questions, but they are disconnected, offer no explainability, mix unverified guesses with confirmed facts, and are often commercial services that require handing over the very identifier the user is worried about.

1.3 Vision and differentiator

The vision is a privacy-first, self-hostable application that accepts an identifier and produces a structured, evidence-backed exposure report. The killer feature is not that the platform runs Sherlock and Holehe; those tools already exist. The differentiator is the intelligence layer built on top of them.

> **Product thesis**
>
> We turn dozens of disconnected OSINT signals into an understandable, evidence-backed, privacy-first digital exposure report. The real intellectual property of this project is the normalization, correlation, confidence, risk, and remediation layer — not the scanners themselves.

Every finding answers six questions: what was found, where it was found, when it was retrieved, how it was discovered, how confident we are, and what the user should do about it. Confidence is never conflated with confirmation, credentials are never displayed, and the entire workflow is gated by an explicit authorization step.

1.4 Primary goals

Goal	Input	Outcome	Primary providers
G1 — Email exposure	user@example.com	Known breaches with dates, organizations, exposed data categories, public references, and potential account associations	XposedOrNot
G2 — Username footprint	username123	Potential accounts across supported platforms with confidence levels	Sherlock, WhatsMyName
G3 — Email-to-account associations	user@example.com	Services where the email appears registered, via safe public signals only	Holehe
G4 — Breach intelligence	email or username	Normalized breach records from at least one provider, optionally two	XposedOrNot, HIBP (optional)
G5 — Explainability	any finding	Evidence, source attribution, timestamps, and confidence for every result	All scanners

Table 1 — MVP goals (what, where, when, how found, confidence, and action)

2. Product Vision, Principles & Non-Goals

2.1 Core principle: self-assessment, not surveillance

The product answers the question of what a person can discover about themselves from publicly accessible or legitimately provided intelligence. It is explicitly not a tool for finding everything about another person. This distinction is not a marketing position; it is built into the architecture. Every scan passes through a consent and authorization gate, domain scanning requires proof of ownership, and the default user interface addresses the person scanning their own identifiers. The workflow below is the canonical product flow and should be treated as a contract between product and engineering.

2.2 Default workflow

```
Canonical scan pipeline

User enters identifier
  -> consent / authorization notice
  -> OSINT scanners
  -> public & breach intelligence
  -> evidence normalization
  -> correlation
  -> confidence scoring
  -> risk analysis
  -> digital footprint report
```

2.3 Design principles

- **Privacy by default.** Privacy Mode never persists identifiers or raw queries, deletes results automatically, and runs no analytics. Local-only deployment keeps every byte on the user's machine.
- **Found is not confirmed.** Every finding carries an explicit confidence level. A username match on a social platform is evidence, never identity verification.
- **Explainability first.** A finding without source, timestamp, scanner version, and confidence is not allowed into the report.
- **Defensive scope only.** The platform never attempts to obtain passwords, bypass authentication or CAPTCHAs, access private accounts, or deanonymize people.
- **Rate-limit discipline.** The system assumes every external provider will throttle it, and never hammers a service because the user clicked Deep Scan.

2.4 Non-goals

The product must never attempt any of the following. These exclusions are permanent product principles rather than deferred features, and future contributions that violate them should be rejected in review.

- Obtain, store, or display passwords, hashes, or raw breach credentials
- Bypass authentication, CAPTCHAs, paywalls, or access controls of any kind
- Access private accounts, private posts, or non-public information
- Exploit websites or perform intrusive scraping of protected resources
- Perform credential stuffing or any form of active attack
- Acquire or redistribute stolen databases
- Deanonymize people using intrusive techniques
- Determine a person's precise physical location
- Claim that an organization illegally used someone's information without evidence

This scope keeps the platform focused on defensive privacy and security. It also keeps the project legally defensible as an open-source effort: every capability shipped in v0.1 reads publicly available information or

uses provider APIs within their published terms.

3. Target Users & MVP Scope

3.1 Personas

Persona	Motivation	Release priority
Individual	*I want to know how exposed my personal information is.*	P0 — primary audience
Security student	*I want to understand OSINT and digital footprinting.*	P0 — primary audience
Developer	*I want an open-source privacy auditing platform I can extend.*	P1 — contributor audience
Security professional	*I want to perform authorized exposure assessments.*	P1
Organization	*I want to monitor employee and domain exposure.*	P2 — domain mode, later phase

Table 2 — Target personas; the first release targets Individual and Security student

The first release deliberately targets individuals and students rather than organizations. Individuals benefit immediately from breach exposure and remediation guidance, while students gain a safe, legal sandbox for understanding how OSINT correlation works. Organizational features such as domain scanning, subdomain enumeration, and employee exposure monitoring require authorization workflows that are planned for a later phase.

3.2 MVP v0.1 scope

The MVP must not try to build everything at once. Version 0.1 supports two identifier types, four scanner integrations, and three output artifacts. Everything else in this document is explicitly deferred to the roadmap in chapter 10.

Dimension	In scope for v0.1	Deferred
Identifiers	Email, username	Phone, domain, full name
Breach providers	XposedOrNot (default)	HIBP behind optional API key
Account discovery	Sherlock, WhatsMyName	Custom platform checks
Email associations	Holehe	Custom public checks
Public web	Not in v0.1	SearXNG integration (Phase 2)
Outputs	Breach report, account discovery, risk score	Identity graph, monitoring, notifications
Deployment	Docker Compose, local-only mode	Cloud multi-tenant offering

Table 3 — MVP scope guardrails

Each scanner integration must degrade independently. If a provider is unreachable, rate-limited, or not configured, the scan continues and the report simply marks that source as unavailable. A missing HIBP key, for example, must never fail an entire scan; the report states that the secondary provider was not configured.

4. System Architecture

4.1 MVP architecture overview

The MVP is a job-based pipeline behind an API gateway. The browser frontend talks to a FastAPI gateway, which validates identifiers, enforces the consent gate, and dispatches scan jobs onto a Redis-backed queue. Dedicated workers wrap each external scanner, emit normalized findings, and hand them to the correlation, risk, and report stages. This design exists because scanners can take significant time and must never block an HTTP request, and because every external provider has distinct rate limits that must be enforced per-worker.

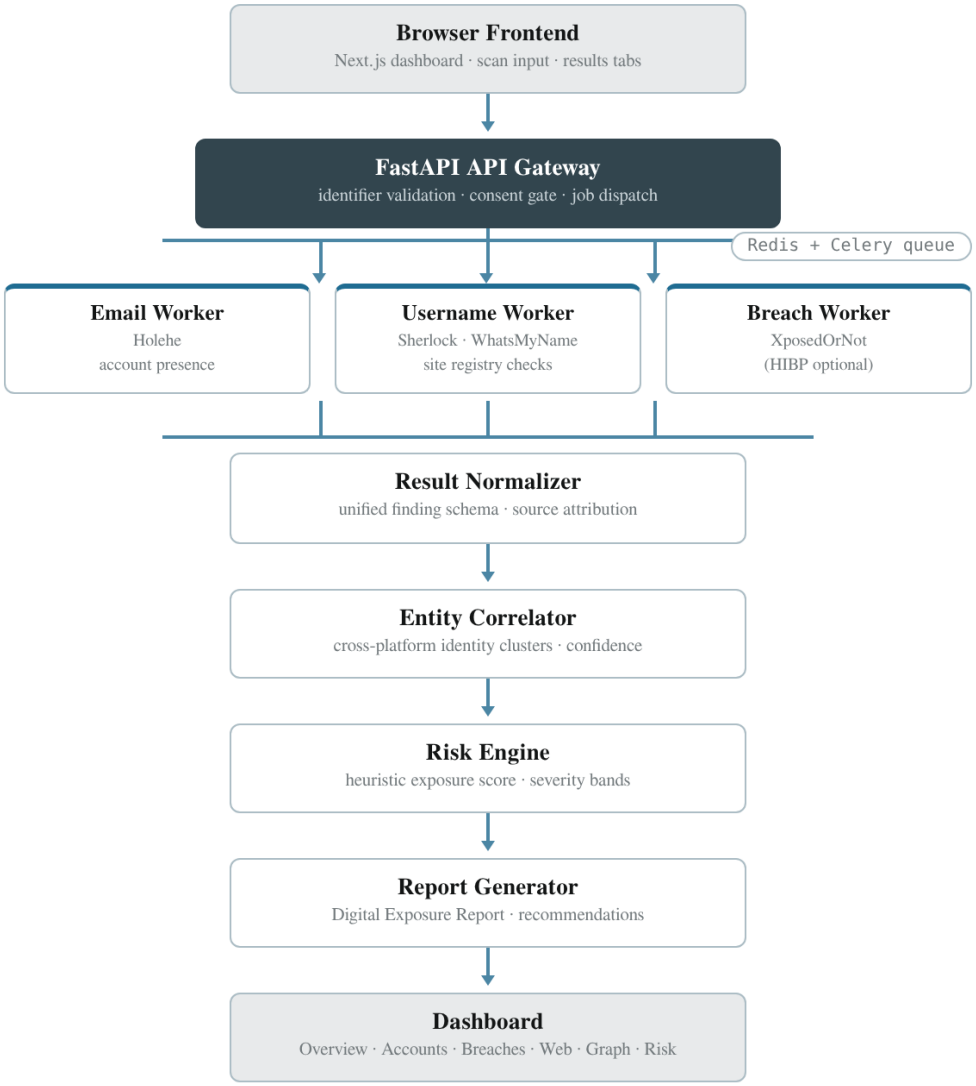


Figure 1 — MVP pipeline: gateway dispatches queued workers; findings flow through normalization, correlation, and risk stages into the report

4.2 Scanner framework

Scanners are never hard-coded into the API. Every integration implements a common asynchronous interface and produces the same output contract, which is the single most important architectural decision in the system. Adding a provider becomes a new module rather than a change to the gateway, and providers with different licensing or rate-limit regimes are isolated behind their adapters.

Scanner contract (Python)

```
class Scanner:
    name: str
    category: str

    async def scan(self, target) -> list[Finding]:
        ...

# Implementations
XposedOrNotScanner   (breach)
HIBPScanner          (breach, optional)
HoleheScanner        (email accounts)
SherlockScanner      (username)
WhatsMyNameScanner   (username)
PublicWebScanner     (web, phase 2)
```

4.3 Unified finding schema

Every scanner output is normalized into one finding schema so the report generator never needs to know which scanner produced a result. The schema carries the finding type, the target type, the source, a confidence value, an explicit status, and a full evidence block with source URL, retrieval timestamp, and scanner version.

Finding schema example (breach)

```
{
  "id": "finding_001",
  "type": "breach",
  "target_type": "email",
  "source": "xposedornot",
  "platform": "ExampleCorp",
  "status": "confirmed",
  "confidence": 1.0,
  "first_seen": "2024-01-10",
  "data_types": ["email", "username"],
  "evidence": {
    "source_url": "https://...",
    "retrieved_at": "2026-10-01T02:10:00Z",
    "scanner_version": "1.2.0"
  }
}
```

```
Finding schema example (social account)
{
  "type": "social_account",
  "target_type": "username",
  "source": "sherlock",
  "platform": "GitHub",
  "username": "example123",
  "url": "https://github.com/example123",
  "status": "potential_match",
  "confidence": 0.92,
  "evidence": { "source_url": "...", "retrieved_at": "..." }
}
```

4.4 Event-driven scanning

Each scanner and pipeline stage emits domain events, which makes the system easy to extend and gives the user interface a truthful progress model instead of an animated loading spinner. The event stream is also the natural substrate for future monitoring and notification features.

```
Scan lifecycle events
SCAN_CREATED
  -> EMAIL_SCAN_STARTED / USERNAME_SCAN_STARTED
  -> BREACH_SCAN_COMPLETED
  -> ACCOUNT_SCAN_COMPLETED
  -> USERNAME_SCAN_COMPLETED
  -> WEB_SCAN_COMPLETED (phase 2)
  -> CORRELATION_COMPLETED
  -> RISK_CALCULATED
  -> SCAN_COMPLETED
```

5. Technology Stack & Licensing

5.1 Frontend

The frontend is Next.js with TypeScript, Tailwind CSS, shadcn/ui component primitives, Recharts for data visualization, and Lucide icons. This combination is fully open source, provides an excellent dashboard ecosystem, deploys easily, and supports strict TypeScript throughout. The dashboard, scan progress console, and report tabs in chapter 8 are all implemented against this stack.

5.2 Backend

The backend is Python 3.12+ with FastAPI, Pydantic models, SQLAlchemy, httpx, and asyncio. Python is a deliberate choice because integration with Holehe, Sherlock, and the wider OSINT and security tooling

ecosystem is dramatically easier than in any alternative. Pydantic enforces the unified finding schema at the boundary, and FastAPI provides asynchronous request handling that suits fan-out scanner dispatch.

5.3 Job system and data stores

Long-running scans are dispatched onto Redis queues processed by Celery workers; one queue per scanner family (email, username, breach, web) so a slow provider cannot starve the others. PostgreSQL stores scans, identifier metadata, findings, sources, confidence values, timestamps, and scan status, while Redis doubles as the cache for rate limiting, duplicate scan suppression, and short-lived results. Elasticsearch is explicitly excluded from the MVP; PostgreSQL is sufficient until the public-web dataset grows, at which point OpenSearch can be introduced. A graph database is likewise deferred: Neo4j would eventually power identity-graph queries, but the MVP models relationships in PostgreSQL.

5.4 Stack summary

Component	Recommendation	Cost / license
Frontend	Next.js + TypeScript	Free, open source
UI kit	Tailwind CSS + shadcn/ui	Free, open source
Backend	FastAPI (Python 3.12+)	Free, open source
Database	PostgreSQL	Free, open source
Queue	Redis + Celery	Free, open source
Breach provider (default)	XposedOrNot	Free tier, rate-limited, no key required
Breach provider (optional)	Have I Been Pwned	API key / subscription required for account search
Username scanning	Sherlock	Free, open source
Username dataset	WhatsMyName	Free, open dataset
Email account checks	Holehe	Free, open source
Public web search	SearXNG (self-hosted)	Free, open source
Graph database (later)	Neo4j Community	Free community edition
Graph UI	React Flow	Free, open source
Charts	Recharts	Free, open source
Containerization	Docker + Docker Compose	Free
Reverse proxy	Caddy	Free, open source

Table 4 — Recommended open-source stack (per component)

5.5 Licensing caveat

Open source does not mean every external data source or API is free for unlimited commercial use. XposedOrNot's basic endpoints need no key but are rate limited, and HIBP's authenticated account-search API requires a paid subscription with visible attribution obligations for breach data. Provider adapters must therefore isolate licensing and rate-limit differences behind their interfaces instead of assuming every source can be freely redistributed. This is the engineering expression of the product's legal posture: the platform is open, the data sources keep their own terms.

6. Scanner & Provider Integrations

6.1 Integration overview

Provider	Role	Access model	Notes
XposedOrNot	Breach provider (default)	No key, free tier	Rate limits approx. 2 req/s, 25/hour, 100/day; Python and JS SDKs published
Have I Been Pwned	Breach provider (optional)	API key / subscription	Authenticated account search requires a key; attribution required for breach data
Holehe	Email account scanner	Open source	Probes password-recovery flows for account presence signals
Sherlock	Username scanner	Open source	Checks a username across 400+ social networks
WhatsMyName	Username site dataset	Open dataset	Structured site registry giving finer control than shelling out to Sherlock
SearXNG	Public web search (Phase 2)	Self-hosted	Legitimate search API for exact-match reference extraction
crt.sh	Certificate transparency (domain mode)	Free public data	Discovers subdomains for authorized domain scans

Table 5 — Provider matrix; every adapter isolates licensing and rate limits

6.2 XposedOrNot (default breach provider)

XposedOrNot is the default breach provider for the open-source edition because its basic email and breach endpoints require no API key, making the default experience zero-configuration. The integration uses the check-email and breach-analytics endpoints, plus the public breach catalog and metrics endpoints for metadata, and normalizes all responses into the unified finding schema. The adapter treats the documented rate limits as hard budget: burst limit 2 requests per second, 25 per hour, and 100 per day, enforced by the Redis rate limiter ahead of every call so the application can never trip a provider limit by accident.

6.3 Have I Been Pwned (optional secondary)

HIBP remains the most comprehensive breach notification service, but its authenticated account-search API requires a key and a subscription. FootprintX therefore treats it as an optional secondary provider. The breach-provider abstraction presents both providers behind one interface; when no HIBP key is configured the scan reports the provider as not configured and continues with XposedOrNot alone. Where HIBP data is used, its attribution requirements under the published license are honored visibly in the report's evidence layer.

6.4 Holehe (email account presence)

Holehe becomes the EmailAccountScanner. Given an email address, it checks whether well-known services recognize the address in their password-recovery flows, producing a safe account presence signal without touching any account content. The output is normalized to a compact finding type, and recovery email or phone fragments observed during probes are never presented as verified identity information.

```
Account presence finding
{
  "type": "account_presence",
  "platform": "instagram",
  "identifier_type": "email",
  "source": "holehe",
  "status": "possible",
  "confidence": 0.8
}
```

6.5 Sherlock and WhatsMyName (username discovery)

Sherlock is the primary username scanner and currently advertises checks across more than 400 social networks, which makes it one discovery engine within the platform rather than the entire application. WhatsMyName is integrated as a structured username-site dataset that feeds an internal site registry, giving the platform more control, better observability, and rate-limit awareness than simply shelling out to an external tool. Both scanners emit social-account findings whose statuses and confidence values are computed by the validation layer rather than trusted blindly from the source.

6.6 Public web and certificate transparency (Phase 2)

The public-web scanner searches for exact identifiers across legitimate search and indexing APIs through a self-hosted SearXNG instance, then extracts, deduplicates, and classifies resulting references. It never aggressively scrapes search engines. For domain-based investigations, certificate transparency logs exposed through crt.sh can reveal subdomains such as api, dev, or legacy hosts under an authorized domain, feeding the organizational exposure mode.

7. Correlation, Confidence & Risk Scoring

7.1 Identity correlation engine

The correlator is the most sophisticated component in the platform. When an email resolves to the same handle on GitHub, Instagram, and Reddit, the engine compares independent signals across those accounts, such as an identical username, a shared external website link, a reused avatar image, and overlapping biography keywords. Each matching signal raises the correlation confidence; each missing signal leaves it untouched. Confidence is then reported as a percentage with the underlying signals shown in full, so the user can see exactly why two accounts were linked.

Account pair	Matching signals	Confidence	Interpretation
GitHub ↔ Reddit	Identical username, same website, same public avatar	93%	Accounts very likely belong to the same person; not verified identity
Instagram ↔ Reddit	Username matches; no shared website, no shared bio	54%	Consistent with the same person, but unverified

Table 6 — Correlation examples; signals are always displayed with the verdict

Correlation never claims ownership. The engine deliberately reports that accounts are likely the same person rather than that they are, and any correlation below the strong threshold is presented with its missing signals visible. This is much safer than asserting that both accounts belong to the scanned person, and it is the mechanism that keeps the platform useful without becoming a deanonymization tool.

7.2 Found is not confirmed

Suppose the scanned username is a common short handle such as alex, and a matching profile exists on a large social network. The system cannot conclude that the profile belongs to the user, and it must say so. Findings therefore use a five-level confidence vocabulary, and confidence is only upgraded on multiple independent signals.

Level	Meaning	Typical evidence
CONFIRMED	The finding is verified by multiple independent signals	Live profile plus corroborating attributes
LIKELY	Strong but unverified match	Live profile with one supporting signal
POSSIBLE	Ambiguous signal requiring review	Existence check returned an uncertain response
NOT FOUND	The scanner found no match at this source	Existence check negative
UNKNOWN	The scanner could not determine a result	Provider unreachable or rate-limited

Table 7 — Confidence vocabulary (a match is never presented as verified identity)

7.3 Risk scoring model

The risk engine computes an Exposure Score from 0 to 100 using a transparent weighted heuristic. Each rule contributes a fixed number of points when triggered, the raw sum is capped at 100, and the final score maps onto five severity bands. The full contribution table is shown to the user in the report so the score is always auditable, rule by rule.

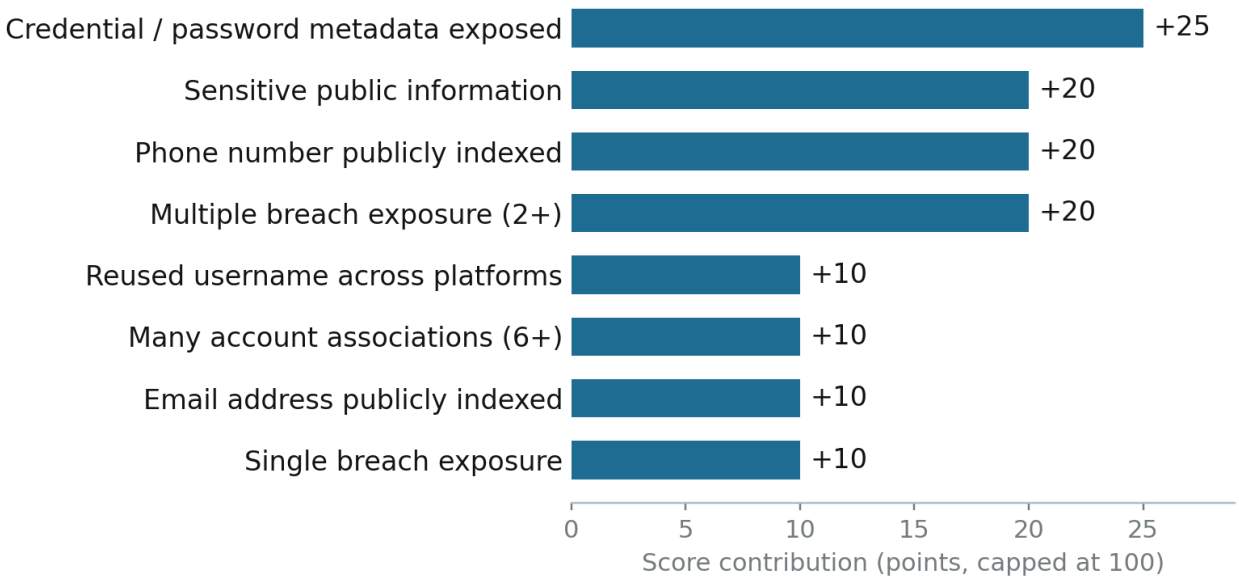


Figure 2 — Exposure score contributions by rule (points, capped at 100)

Band	Score range	Reading
LOW	0 – 20	Minimal public exposure
MODERATE	21 – 40	Some exposure worth cleaning up
ELEVATED	41 – 60	Meaningful exposure; act on top findings
HIGH	61 – 80	Significant exposure; remediation recommended now
CRITICAL	81 – 100	Heavy exposure across multiple categories

Table 8 — Severity bands for the 0–100 exposure score

> **Scoring disclaimer**
>
> The Exposure Score is the application's own weighted heuristic, not an industry-standard security score. It measures observable public exposure, not actual compromise, and the report states this alongside the score at all times.

8. Product Experience

8.1 Scan lifecycle and progress

The home screen is a single field for an email or username, a scan-mode selector, and the start action behind a consent dialog. During a scan, the interface never shows a bare loading spinner; it shows the staged pipeline with completed, active, and pending stages, plus a live event stream of scanner events. This truthful progress model exists because the architecture genuinely is event-driven, so the interface can simply replay reality.

```
Progress states shown to the user
Scanning digital footprint
  [done]  Validating identifier
  [done]  Checking breach databases
  [done]  Checking username associations
  [active] Checking social platforms
  [pending] Correlating results
  [pending] Calculating exposure score
```

8.2 Results dashboard

The results dashboard opens with the exposure score, four summary counters for breaches, accounts, mentions, and triggered risks, and a tab set covering every report dimension: Overview, Accounts, Breaches, Public Web, Identity Graph, Risk, Recommendations, and Evidence. The Identity Graph tab is a Phase 7 preview rendered with React Flow, showing the identifier at the center, discovered accounts around it,

shared attributes such as a website or avatar, and correlation edges labeled with their confidence.

8.3 Breach presentation rules

Each breach card names the organization, the breach date, the affected record count, the exposed data categories, a severity rating, and a short remediation checklist. Passwords and hashes are never displayed, not even partially, and the interface states this rule explicitly so users understand what they are seeing.

> **Breach UI contract**
>
> Breach cards report metadata only: organization, date, and exposed data categories. Never display password values, password hashes, or raw credentials in any form. Remediation guidance replaces sensation with action: change reused passwords, enable MFA, and review related accounts.

8.4 Accounts and public exposure

The Accounts tab lists every discovered account with its platform, profile URL, discovery source, status, and a confidence indicator, so a Reddit account found with 63 percent confidence is visibly different from a GitHub account at 96 percent. The Public Web tab groups references by kind, such as repositories, forums, websites, and documents, with each reference showing its URL, date, and a short snippet of context.

8.5 Recommendations engine

The report never simply says that the user is exposed; it pairs every finding category with concrete actions. Breach findings produce an action-required card covering password reuse, MFA, and session review. Indexed personal details produce a privacy-improvement card covering profile cleanup and search-engine removal options. Each recommendation is a checklist the user can work through, which is also how the platform measures its own usefulness without collecting analytics.

8.6 Scan modes

Mode	Inputs	Scanners	Notes
Quick scan	Email or username	Breach + basic account discovery	Fast, roughly a minute of wall time
Deep scan	Email and username	Breach + social + public web + correlation	Adds SearXNG reference extraction and identity clusters
Domain scan	Domain (authorization required)	DNS + subdomains + emails + breaches	Requires proof of ownership or explicit authorization

Table 9 — Scan modes

8.7 Privacy Mode

Privacy Mode is the flagship privacy differentiator. When enabled, identifiers are not persisted, raw search queries are not stored, results are deleted automatically at the end of the session, and no analytics of any kind are collected. The user chooses a retention policy before every scan, and the choice is honored by the storage layer rather than by a user-interface promise.

Retention option	Behavior
Save for 30 days	Report stored; identifier kept hashed and masked; automatic purge after 30 days
Save until deleted	Report stored until the user removes it; identifier hashed and masked
Delete after scan (Privacy Mode)	Nothing persisted; results live in memory for the session only

Table 10 — Retention options offered before every scan

8.8 Local-only mode

A single Docker Compose command starts the frontend, API, workers, Redis, and PostgreSQL on the user's own machine, with the dashboard reachable at localhost. Local-only deployment minimizes trust requirements: no third party ever sees the identifiers the user scans, which is the strongest privacy statement the platform can make. The same compose file optionally adds a SearXNG service when the public-web phase ships.

9. API, Data & Security Design

9.1 API surface

Method	Path	Purpose
POST	/api/v1/scans	Start a scan; validates and queues the identifier
GET	/api/v1/scans/{scan_id}	Scan status, progress, and event stream
GET	/api/v1/scans/{scan_id}/results	Full digital exposure report
GET	/api/v1/scans/{scan_id}/findings	Raw normalized findings
GET	/api/v1/scans/{scan_id}/risk	Exposure score and contributions
DELETE	/api/v1/scans/{scan_id}	Delete a scan and its stored findings
GET	/api/v1/scanners	Scanner health and configuration status

Table 11 — Public API surface (versioned under /api/v1)

```
Start scan request and response
POST /api/v1/scans
{ "type": "email", "value": "user@example.com", "mode": "deep" }

201 Created
{ "scan_id": "scan_123", "status": "queued" }
```

9.2 Data model and identifier handling

PostgreSQL stores the scan record, identifier metadata, findings, sources, confidence values, timestamps, and status. The platform stores no more than necessary: where practical, only the identifier hash is persisted, with the raw identifier reserved for short-lived worker state. Masked display forms such as a partially redacted email are derived at render time so history remains useful without retaining plaintext identifiers.

9.3 Source attribution

Every result retains its source, source URL, retrieval timestamp, and scanner version, and this attribution travels with the finding into every export. Attribution is a correctness requirement, not a courtesy: when multiple breach providers are combined, the report must be able to state exactly which provider produced which record, and HIBP data specifically requires visible attribution under its license terms.

9.4 Rate limiting

The architecture assumes every scanner will be rate limited. A Redis-backed limiter sits ahead of every provider call, enforcing per-provider budgets derived from their published limits, including XposedOrNot's 2 requests per second, 25 per hour, and 100 per day. Deep scans queue patiently rather than burst, and limit state is exposed through the scanner health endpoint so operators can see how close the platform is to any budget.

9.5 Scanner health system

The scanner health endpoint reports each provider's operational state, last successful scan, last error, response time, and rate-limit state. HIBP without a key shows as not configured rather than failed, and Phase 2 scanners appear as not yet enabled. This surface exists so both users and operators can distinguish a slow provider from a broken one, and it is rendered directly in the dashboard header.

9.6 Security requirements

All network traffic uses TLS. Stored scan data and secrets are encrypted at rest, and secrets live in a secret manager in production or in a git-ignored environment file locally; API keys and database passwords are never committed. The MVP launches with no accounts and ephemeral scans, which deliberately minimizes the personal information the platform itself holds. Cloud deployments may later add OAuth or email login, but the default posture remains collect-nothing.

9.7 Audit logging

Logged	Never logged
Scan started and completed events	Full email addresses or phone numbers
Scanner errors and rate-limit events	Raw sensitive finding contents
System errors with sanitized context	Any raw identifier where a hash suffices
Identifier hashes for correlation	Source URLs tied to raw identifiers

Table 12 — Audit logging rules; the hash replaces the identifier wherever practical

10. Delivery Roadmap

10.1 Phased plan

Phase	Window	Deliverable	Exit criterion
0 — Foundation	Week 1	Repository, Docker, FastAPI, Next.js, PostgreSQL, Redis, CI	Compose-up runs the empty pipeline end to end
1 — Breach engine	Week 2	XposedOrNot integration and normalized schema	Email in, breach report out
2 — Username engine	Week 3	Sherlock and WhatsMyName integrations	Username in, potential accounts out
3 — Email accounts	Week 4	Holehe integration	Email in, associated services out
4 — Correlation	Week 5	Entity model, confidence engine, dedup, evidence model	Accounts correlate with visible confidence
5 — Dashboard	Week 6	Overview, breaches, accounts, evidence, risk, recommendations	Full report usable by a non-technical user
6 — Public web	Weeks 7–8	SearXNG, URL extraction, document detection	Deep scans include public references
7 — Graph	Later	Neo4j and React Flow identity graph	Interactive graph over real scan data
8 — Monitoring	Later	Scheduled scans and breach notifications	Opt-in notifications delivered on new exposure

Table 13 — Phase roadmap; phases 0–6 form the v0.1 release train

10.2 Monitoring and notifications

Phase 8 introduces scheduled monitoring: a daily scan for watched identifiers, with notifications when a new breach, a new account, or new public exposure appears. Delivery channels are email, Telegram, and Discord webhooks, and every channel requires explicit opt-in before a single message is sent. The retention and privacy guarantees of chapter 8 apply unchanged to monitored identifiers, and users can delete watch rules at any time.

10.3 v1.0 definition

Version 1.0 is the union of the email, username, and phone pipelines feeding a shared breach-intelligence stage, with privacy mode and local-only deployment as first-class citizens. Phone intelligence and domain intelligence land in this release along with the graph and monitoring phases. The release criterion is not a feature count; it is the quality of the intelligence layer.

> **v1.0 success criterion**
>
> The killer feature is not that we run Sherlock and Holehe — those tools already exist. Version 1.0 succeeds when dozens of disconnected OSINT signals become one understandable, evidence-backed, privacy-first exposure report that a non-expert can act on. Normalization, correlation, confidence, risk, and remediation are the product; the scanners are its inputs.